

onod: A Full-Rust CPU-Only Hybrid Search Engine with ONNX Transformer Embeddings and Sparse Fusion for 100% Multilingual Recall at 54 ms

Yudono Putro

Independent Research, Jakarta, Indonesia

September 2026

Abstract—onod is a search engine for multilingual documents written entirely in Rust. It runs on commodity CPUs with no GPU. Every 600-character chunk is encoded twice. The dense code comes from a multilingual Transformer exported to ONNX (384 dimensions, int8 quantization, a 250k Unigram tokenizer cut at 128 tokens); we mean-pool *last_hidden_state* over the attention mask and L2-normalize. The sparse code is a TF-IDF vector over roughly 30k terms. The two rankings merge through reciprocal rank fusion (0.6 dense, 0.4 sparse, $k=60$), adjusted by a boost for financial vocabulary, by character-trigram overlap measured on the top 200 candidates, and by a final cosine rerank. We measured the system on three financial PDFs totalling 4.3 MB (3,750 chunks) on an eight-core Apple M2 with 16 GB of memory. It retrieves a matching chunk for all 20 test queries in English, Indonesian, and Chinese — 100% recall — at 54.0 ms mean latency, ranging from 45.4 to 79.7 ms against a 100 ms budget. Indexing the same corpus costs about 100 seconds on CPU because inference still runs one chunk at a time; we return to that bottleneck in Section 7. When *model.onnx* is missing, onod degrades to hashed character n -gram vectors instead of failing.

Index Terms—hybrid retrieval, ONNX Runtime, multilingual embeddings, RRF fusion, Rust, CPU-only.

1 Introduction

Retrieval pipelines usually encode each chunk with a model from the BERT family: MiniLM, LaBSE, and more recently BGE-M3 or multilingual-E5 [3]–[7]. At gigabyte scale in many languages, that means millions of forward passes before the first query can run. On a CPU machine this easily passes 60 seconds of ingestion, which is why teams reach for GPUs. Dense vectors have a second weakness that matters for finance: an exact string such as 62.714.280 can drown in embedding noise, while plain BM25 [1] cannot see that *revenue*, *pendapatan*, and a Chinese paraphrase mean the same thing.

onod refuses that tradeoff. Each chunk gets a dense Transformer vector computed on CPU through ONNX Runtime [11] in int8, with a sparse TF-IDF vector beside it. The two rankings merge through RRF [2], which needs no score calibration, followed by a finance boost, a trigram overlap pass, and a cosine rerank that reuses vectors already in memory. Nothing in the query path needs a GPU, a Python server, or a separate index per language.

Three things in this paper are new as far as we know. First, a dense ONNX path that runs fully on CPU: three inputs, masked mean pooling, normalized output, the query encoded exactly like the documents, and a hash fallback when the model file is absent. Second, the whole hybrid in Rust — dense plus sparse plus fusion plus boosts plus rerank in one binary. Third, a small trilingual benchmark of 20 queries where the system scores 100% recall at 54 ms, plus an honest account of where the remaining time goes: CPU indexing at roughly 100 seconds for 3,750 chunks, and a plan to batch it away.

2 System Architecture

Documents arrive as PDF through *pdf_oxide* or as plain text, Markdown, CSV, and HTML. Normalization is Unicode NFKC with Arabic diacritics stripped and whitespace collapsed. A sentence splitter that understands CJK terminators feeds chunks of about 600 characters with a 140-character overlap; table rows past 900 characters are cut at word boundaries. Headings in all caps or starting with BAB or Chapter survive as metadata.

2.1 Dense path (ONNX Transformer)

The dense code starts from a 250k Unigram tokenizer (whitespace splitting plus metaspaces, the XLM-R recipe [5]) truncated to 128 tokens longest-first, padding id 1. The ONNX graph takes *input_ids*, *attention_mask*, and *token_type_ids* shaped [batch, seq] and returns *last_hidden_state* shaped [batch, seq, 384]; the weights are a paraphrase-multilingual-MiniLM variant quantized to int8 for ARM64, 113 MB on disk. Pooling is

$$\mathbf{v} = \frac{\sum_i m_i \mathbf{h}_i}{\sum_i m_i}, \quad \hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\| \quad (1)$$

where m_i masks padding and $\mathbf{h}_i$ are the hidden states. Queries travel the identical path as documents: *embed_query* delegates to *embed* whenever a model is loaded, so the two sides cannot disagree. If tokenization or inference fails for any reason, that chunk gets a hash embedding instead. The session sits behind *Mutex<Session>* because *Session::run* takes *&mut*.

2.2 Sparse path and fusion

The sparse side is TF-IDF over words that appear at least twice, stored sparsely. Its ranking and the dense ranking

merge as

$$\text{RRF}(d) = \frac{0.6}{60 + \text{rank}_{\text{dense}}(d)} + \frac{0.4}{60 + \text{rank}_{\text{sparse}}(d)} \quad (2)$$

A chunk then earns +0.3 when it carries finance vocabulary together with more than 20 digits, and up to +2.0 times its character-trigram overlap with the query — measured only on the top 200 candidates so the cost stays flat — before a final rerank of half fusion score, half $\cosine(q_{\text{dense}}, d_{\text{dense}})$ that reuses vectors already in memory.

3 CPU-Only Execution

The *ort* crate (2.0.0-rc.13) runs with its default CPU execution provider [11]; *tokenizers* 0.21 and *ndarray* 0.17 build the [1, seq] int64 tensors. There is no CUDA, no CoreML, and no Python at serving time. Scoring — dense cosine plus sparse intersection — runs over *rayon* parallel iterators on all eight cores, PDF pages parse on threads, and files over 100 MB are memory-mapped. Chunk inference is still serial behind the mutex at about 27 ms a chunk. ORT intra-op threads do spread each run over several cores, but end-to-end throughput sits near 37 chunks a second. Batching 8 to 16 chunks per run and giving each thread its own session should remove the bottleneck.

4 Benchmark Results

All numbers below come from an Apple M2 with eight cores and 16 GB, release build. The corpus in *files/* is the ANTAM financial statement of 30 June 2026 (1.6 MB, 1,909 chunks), Adaro (2.7 MB, 1,612), and 10840 (25 MB, 229): 3,750 chunks in total. The queries live in *tests/test_20.txt* across English, Indonesian, and Chinese. Our metric is deliberately simple: a hit means one of the expected keywords shows up in the top-10 snippets or their expansions, in the spirit of BEIR hit-rate [12]. Full nDCG/MRR judgments in the style of MIRACL [13] are future work.

4.1 Accuracy: 20/20

#	Query (short)	ms	Hit
1	total uang dari pelanggan	79.7	OK
2	pendapatan bersih ANTAM S1	66.6	OK
3	money from contracts (EN)	62.7	OK
4	selisih pendapatan–beban	65.4	OK
5	keuntungan sbml operasi	63.3	OK
6	profit after tax	63.1	OK
7	nilai kekayaan perusahaan	53.3	OK
8	kewajiban harus dibayar	53.3	OK
9	bagian pemegang saham	50.8	OK
10	keuntungan dibagikan	48.3	OK
11	kas dari operasi	52.9	OK
12	jumlah emas diproduksi	48.1	OK
13	produksi nikel (ton)	45.4	OK
14	presiden direktur?	45.9	OK
15	produk tambang utama	46.9	OK
16	beban pajak penghasilan	47.2	OK
17	piutang pelanggan	46.4	OK
18	stok emas blm dijual	46.6	OK
19	selisih pendapatan–beban	46.8	OK
20	ANTAM 2026 H1 revenue (CN)	47.5	OK

Table 1: Twenty queries, twenty hits — including the Chinese Q20, which the hash fallback misses.

4.2 Performance

Metric	Value	Target
Recall (20 queries)	100%	>95%
Avg query latency	54.0 ms	<100 ms
Min / max query	45.4 / 79.7 ms	<100 ms
Index (3750 chunks)	~100 s	<60 s
Throughput	~37 chunk/s	—
Embedding dim	384	—
GPU	none (CPU)	CPU-only

Table 2: The query path meets the 100 ms budget; indexing misses the 60 s one (serial inference).

4.3 Scaling

Corpus	Chunks	Share
ANTAM (1.6 MB)	1909	51%
Adaro (2.7 MB)	1612	43%
10840 (25 MB)	229	6%
Combined	3750	100%

Table 3: Where the chunks come from; indexing cost grows linearly with chunk count.

5 Comparison with Baselines

System	Recall	Query	Index
Hash fallback	19/20	~2 ms	~2–8 s
onod+ORT CPU	20/20	~54 ms	~100 s
Dense MiniLM Py	~80%	~55 ms	~15 s*
SPLADE [8]	100%†	~1 ms	~4 s†

Table 4: Runs marked * used a GPU; the SPLADE number is a stub measured on ANTAM (†). The ONNX build wins recall and pays for it at index time.

Plain BM25 [1] answers keyword queries in about a millisecond and misses every paraphrase. A dense-only MiniLM [4] wants a GPU at index time and still drops exact numbers. SPLADE [8] and ColBERTv2 [9] are excellent but heavier to serve. onod keeps the BM25 habit of exact matching through its sparse side and number boost, and adds cross-lingual reach through a dense side that borrows ideas from the BGE-M3 [6] and E5 [7] families, distilled into a 384-dimensional MiniLM — with no vector database and no GPU.

6 Implementation Details

The engine is about 790 lines of Rust in *core/src/main.rs*: a *TransformerEmbedder* holding a *Mutex<Option<Session>>* plus the tokenizer, a *SparseEmbedder*, a *Reranker*, NFKC normalization, sentence-aware chunking, an *OnodIndex* serialized with bincode to *index.bin*, subcommands for benchmark, search, and serve, and a small REST server from the standard library. Dependencies are *ort*, *tokenizers*, *ndarray*, *rayon*, *unicode-normalization*, *memmap2*, *serde*, and *pdf_oxide*. Three bugs in the first prototype deserve mention because each one broke retrieval silently or loudly: the tokenizer was an empty WordPiece that panicked with *MissingUnkToken*, so it now loads the real *tokenizer.json*; the session was fed two inputs while the graph needs three, hence the zeros in *token_type_ids*; the output was sliced raw with *take(dim)* instead of pooled; and queries were hashed while documents went through the model, so *embed_query* now routes both sides through the same function.

7 Limitations and Future Work

Four limits are worth stating plainly. Indexing costs around 100 seconds, past the 60-second budget, because of the serial mutex around inference. The trigram structures use real memory. The metric is keyword hits, not nDCG or MRR on MIRACL [13] or BEIR [12]. And truncating every chunk at 128 tokens can clip long tables. The order of work follows from that list: batch 8–16 chunks per inference run, hand each thread its own session, prune with WAND or block-max WAND [15] before reranking the top 200 down to 20, make the dense index persistent and incremental, evaluate on MIRACL, Mr. TyDi, and multilingual BEIR, and optionally rerank candidates with a cross-encoder [10].

8 Conclusion

A sparse engine with a small dense side can carry trilingual finance search on a CPU: 20 out of 20 queries at 54 ms mean latency, no GPU involved. Indexing

throughput is the open item, and batching plus session parallelism should close it without touching the sub-100 ms query path.

References

- [1] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, 2009.
- [2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *SIGIR*, 2009.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *EMNLP*, 2019.
- [4] N. Reimers and I. Gurevych, “Making monolingual sentence embeddings multilingual using knowledge distillation,” 2020.
- [5] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” in *ACL*, 2020.
- [6] J. Chen et al., “BGE-M3: Multi-lingual, multi-functionality, multi-granularity text embeddings,” 2024.
- [7] L. Wang et al., “Multilingual E5 text embeddings,” 2024.
- [8] T. Formal et al., “SPLADE v2: Sparse lexical and expansion model for information retrieval,” 2021.
- [9] K. Santhanam et al., “ColBERTv2: Effective and efficient retrieval via lightweight late interaction,” in *SIGIR*, 2022.
- [10] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *EMNLP*, 2020.
- [11] ONNX Runtime Developers, “ONNX Runtime: CPU execution provider, quantization and performance,” 2024.
- [12] N. Thakur et al., “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *NeurIPS Datasets*, 2021.
- [13] X. Zhang et al., “MIRACL: Multilingual information retrieval across a continuum of languages,” 2023.
- [14] J. Lin et al., “Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations,” in *SIGIR*, 2021.
- [15] A. Broder et al., “Efficient query evaluation using a two-level retrieval process,” in *CIKM*, 2003.

Appendix A Reproducibility

One-command CPU benchmark (Apple M2, eight cores):

```
cd core
# place model.onnx (113M qint8) + tokenizer.json
# (8.7M) in core/ (git-ignored)
cargo build --release
./target/release/onod benchmark ../files/
# expect: ORT model loaded, 3750 chunks, 20/20 =
100%, avg ~54ms
```

Sources: *core/src/main.rs* (ORT+hybrid), *core/src/lib.rs* (FFI BM25), *tests/test_20.txt* (20 queries), *benchmarks/ANTAM_benchmark.md* (Python baselines), *files/* (3 PDFs). Model files (*.onnx, tokenizer.json) are git-ignored.